

به نام خدا

درس ابزار دقیق

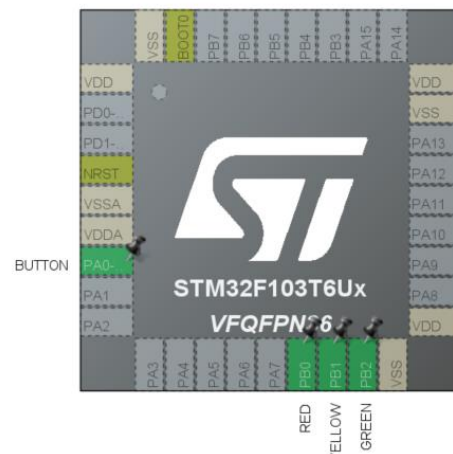
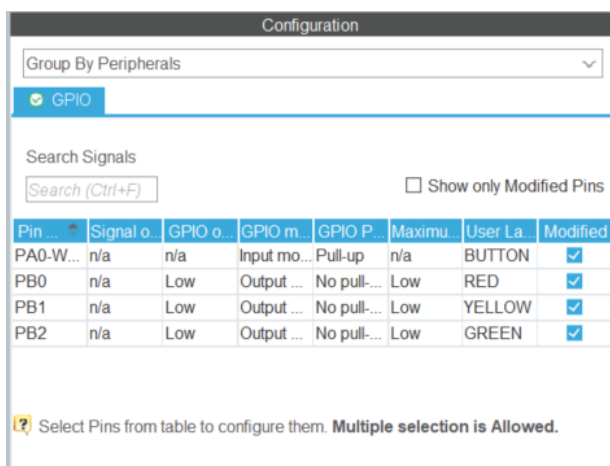
تمرین سری سوم

متن شایسی 810101453

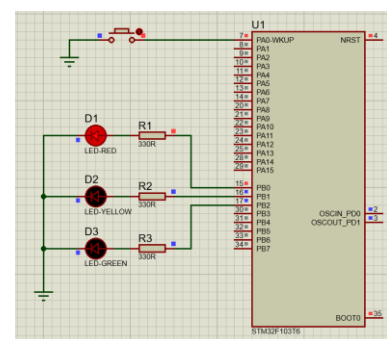
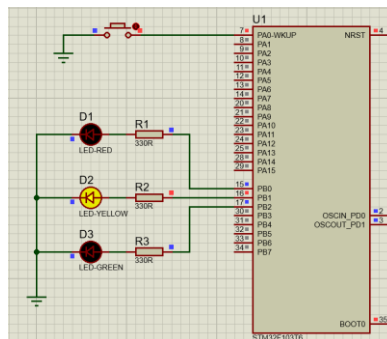
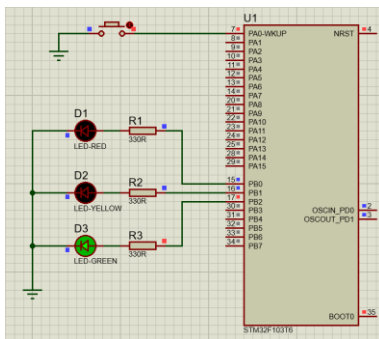


سوال 1) یک پین از GPIO پورت A را به عنوان ورودی برای کلید و سه پین دیگر از پورت A را به عنوان خروجی برای اتصال به LED ها انتخاب میکنیم. در تنظیمات دکمه ورودی باید آنرا به صورت pull-up در نظر بگیریم. همچنین مقدار کلاک اصلی میکروکنترلر را 36MHz میگذاریم.

$$SID \% 3 = 2 \rightarrow STM32F103T6, \quad SID \% 9 = 5 \rightarrow 36MHz$$

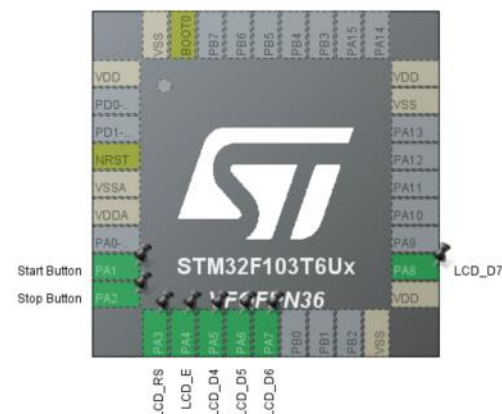


در ادامه پس از Generate code و اضافه کردن کدهایی که در ادامه آورده شده است میتوان فایل Q1.hex را ساخت که باید آن را در میکروکنترلر برنامه Proteus لود کرد و سپس مدار را به طور زیر با توجه به پین هایی که ست کردیم اتصالات را انجام دهیم تا هر سری با فشردن push button که یک pull-up است یکی از سه چراغ روشن شود.



سوال 2) در این سوال پایه ها طبق شکل زیر مشخص شده اند که PA1 و PA2 کلید های شروع و توقف هستند که بصورت pull-down تعریف شده اند. سایر پایه ها برای اتصال به LCD میباشند. در ادامه طبق خواسته interrupt ها را فعال میکنیم چراکه باید هر ثانیه فعال شده تا پس از overflow سراغ اجرای تابع HAL_TIM_Base_Start_IT برود.

EXTI line1 interrupt	☒
EXTI line2 interrupt	☒
TIM1 break interrupt	☐
TIM1 update interrupt	☒
TIM1 trigger and commutation interrupts	☐
TIM1 capture compare interrupt	☐
TIM2 global interrupt	☒



Configure the below parameters :

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value) 35999

Counter Mode Up

Counter Period (AutoReload ...) 999

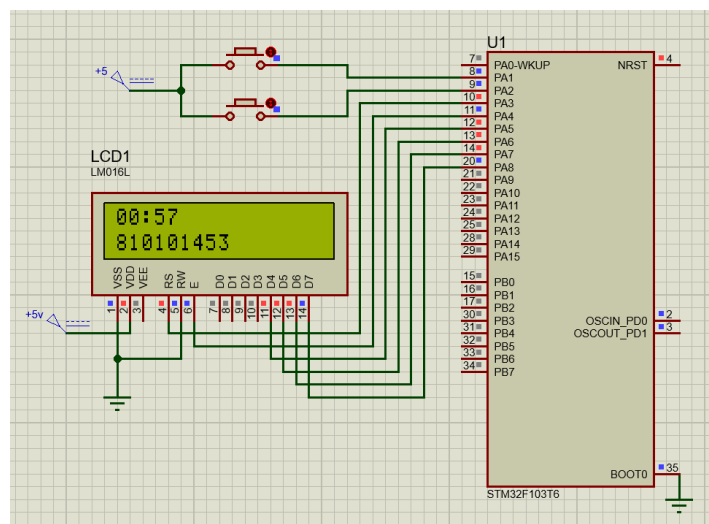
Internal Clock Division (CKD) No Division

auto-reload preload Disable

با کلاک 36MHz، اگر Prescaler = 35999 و Period = 999 باشد، تایمر هر 1 ثانیه یکبار وقفه ایجاد میکند. چون Prescaler فرکانس را به ۱ kHz می‌رساند و Period شمارش را به ۱۰۰۰ تیک تنظیم می‌کند. درواقع Prescaler کاری می‌کند که تایمر کند شود، Period تعیین می‌کند هر چند تا تیک یکبار وقفه بزند.

$$\text{Timer Update Frequency} = \frac{F_{CLK}}{(period + 1) * (Prescaler + 1)}$$

در نهایت خروجی hex را میگیریم و مدار را در proteus میبندیم تا بصورت زیر شود:

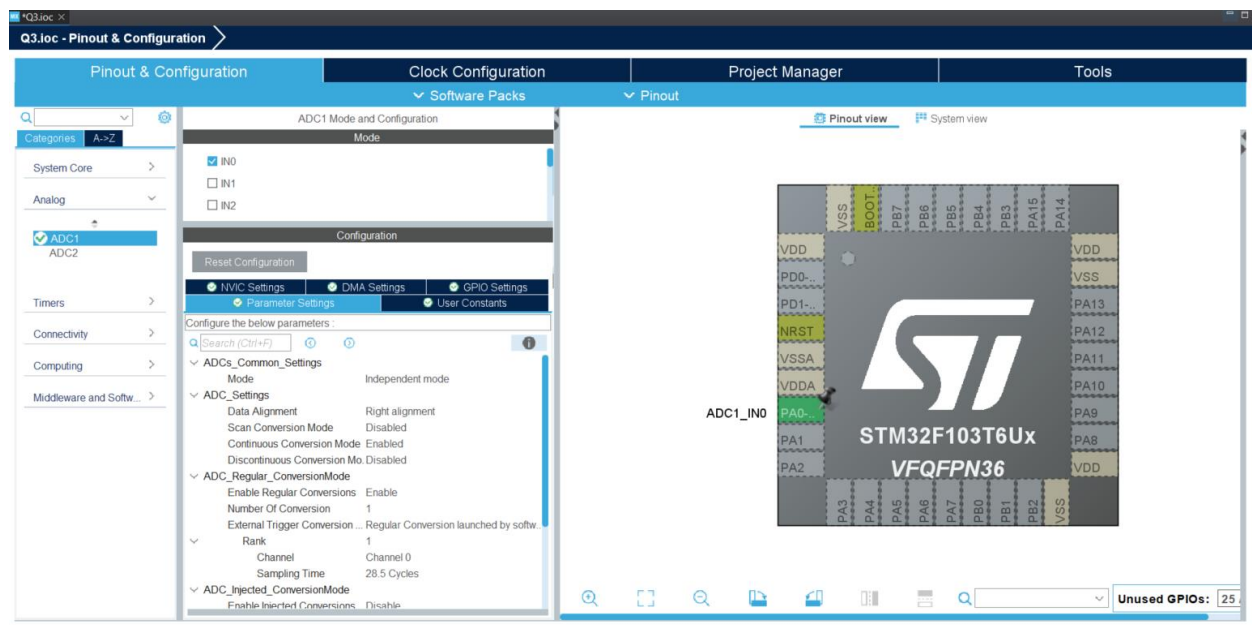


سوال 3

الف) ADC یا Analog to Digital Converter واحدی در میکروکنترلر است که ورودی آنالوگ (مثل ولتاژ سینوسی یا سیگنال از سنسور) را به داده دیجیتال تبدیل می‌کند تا میکرو بتواند با آن کار کند. مثال: اگر ولتاژ بین 0 تا 3.3 ولت باشد و مبدل 12 بیت باشد، خروجی دیجیتال عددی بین 0 تا 4095 خواهد بود.

ویژگی‌های آن: دقتی معمولاً 12 بیتی دارد. (قابل تنظیم به 10 و 8 و 6 بیت) – میتواند چند کاناله باشد – امکان کار به صورت triggered و continuous mode را دارد

برای اضافه کردن آن به stm32 هم بطور زیر عمل میکنیم:



پایه‌ای مانند PA0 را برای ورودی آنالوگ ADC1_IN0 انتخاب میکنیم. در بخش تنظیمات ADC، گزینه‌هایی مانند رزولوشن را روی 12 بیت قرار میدهم، "Scan Conversion Mode" را غیرفعال و "Continuous Conversion Mode" را فعال میکنیم تا ADC به طور پیوسته مقادیر آنالوگ را بخواند. سپس از تب "Rank" کانال IN0 را انتخاب کرده و زمان نمونه‌برداری را تنظیم میکنیم. در ادامه کار باید در کد main.c از دستورات HAL_ADC_Start(&hadc1) و HAL_ADC_PollForConversion و HAL_ADC_GetValue استفاده کنیم تا مقدار دیجیتال خروجی ADC را دریافت کنیم.

ب) در ادامه خواسته شده تا برنامه‌ای برای نمایش رو LCD بنویسیم که پس از Generate code کردن تنظیمات و اضافه کردن دستورات خودمان به کدی که در زیر آمده میرسیم:

```

void LCD_Delay(uint32_t time) {
    for (uint32_t i = 0; i < time * 10; i++) __NOP();
}

void LCD_EnablePulse(void) {
    HAL_GPIO_WritePin(LCD_Port, EN_Pin, GPIO_PIN_SET);
    LCD_Delay(2);
    HAL_GPIO_WritePin(LCD_Port, EN_Pin, GPIO_PIN_RESET);
    LCD_Delay(2);
}

void LCD_Send4Bits(uint8_t data) {
    HAL_GPIO_WritePin(LCD_Port, D4_Pin, (data >> 0) & 1);
    HAL_GPIO_WritePin(LCD_Port, D5_Pin, (data >> 1) & 1);
    HAL_GPIO_WritePin(LCD_Port, D6_Pin, (data >> 2) & 1);
    HAL_GPIO_WritePin(LCD_Port, D7_Pin, (data >> 3) & 1);
    LCD_EnablePulse();
}

void LCD_SendCommand(uint8_t cmd) {
    HAL_GPIO_WritePin(LCD_Port, RS_Pin, GPIO_PIN_RESET);
    LCD_Send4Bits(cmd >> 4);
    LCD_Send4Bits(cmd & 0x0F);
    LCD_Delay(5);
}

void LCD_SendData(uint8_t data) {
    HAL_GPIO_WritePin(LCD_Port, RS_Pin, GPIO_PIN_SET);
    LCD_Send4Bits(data >> 4);
    LCD_Send4Bits(data & 0x0F);
    LCD_Delay(5);
}

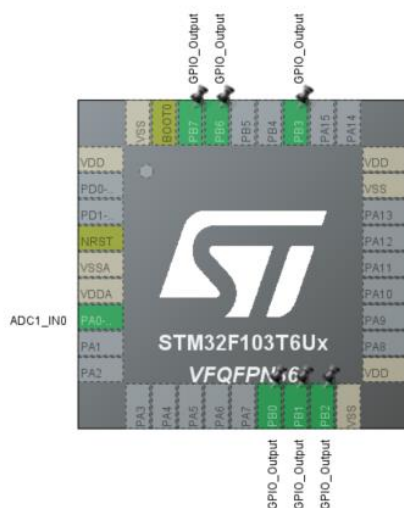
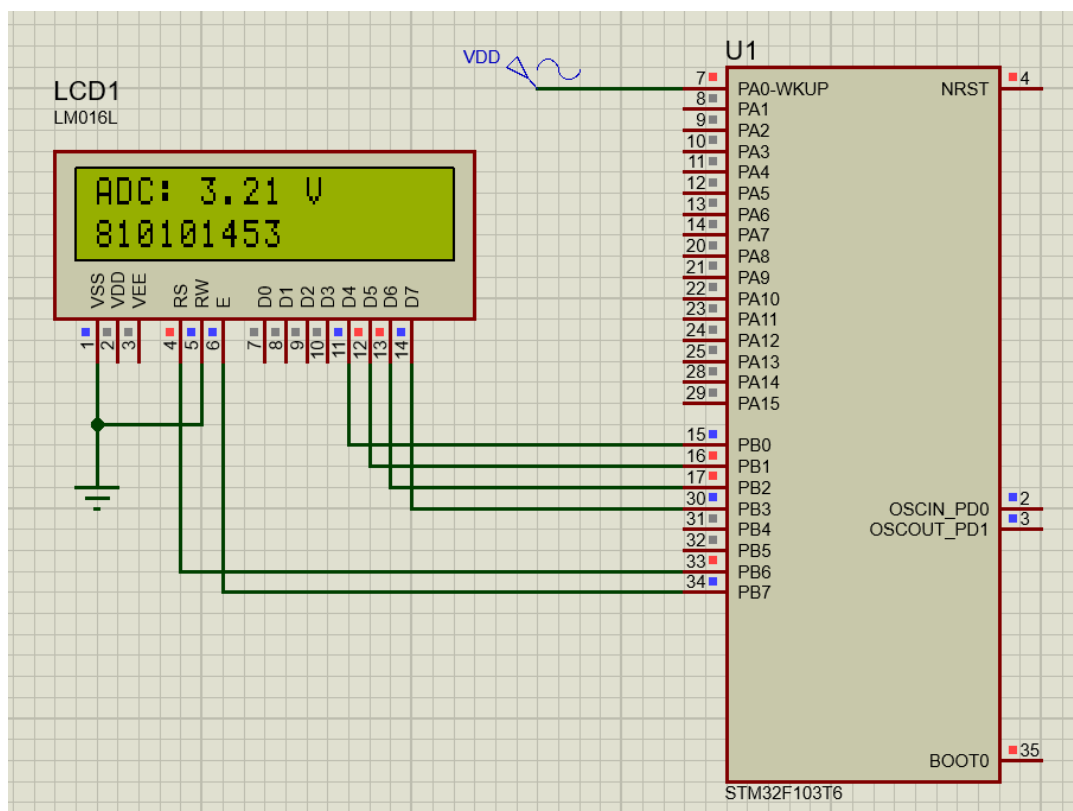
void LCD_SendString(char* str) {
    while (*str) LCD_SendData(*str++);
}

void LCD_SetCursor(uint8_t row, uint8_t col) {
    uint8_t pos = (row == 0) ? (0x80 + col) : (0xC0 + col);
    LCD_SendCommand(pos);
}

void LCD_Init(void) {
    LCD_Delay(500);
    LCD_Send4Bits(0x03); LCD_Delay(50);
    LCD_Send4Bits(0x03); LCD_Delay(50);
    LCD_Send4Bits(0x03); LCD_Delay(50);
    LCD_Send4Bits(0x02);
    LCD_SendCommand(0x28);
    LCD_SendCommand(0x0C);
    LCD_SendCommand(0x06);
    LCD_SendCommand(0x01);
}

```

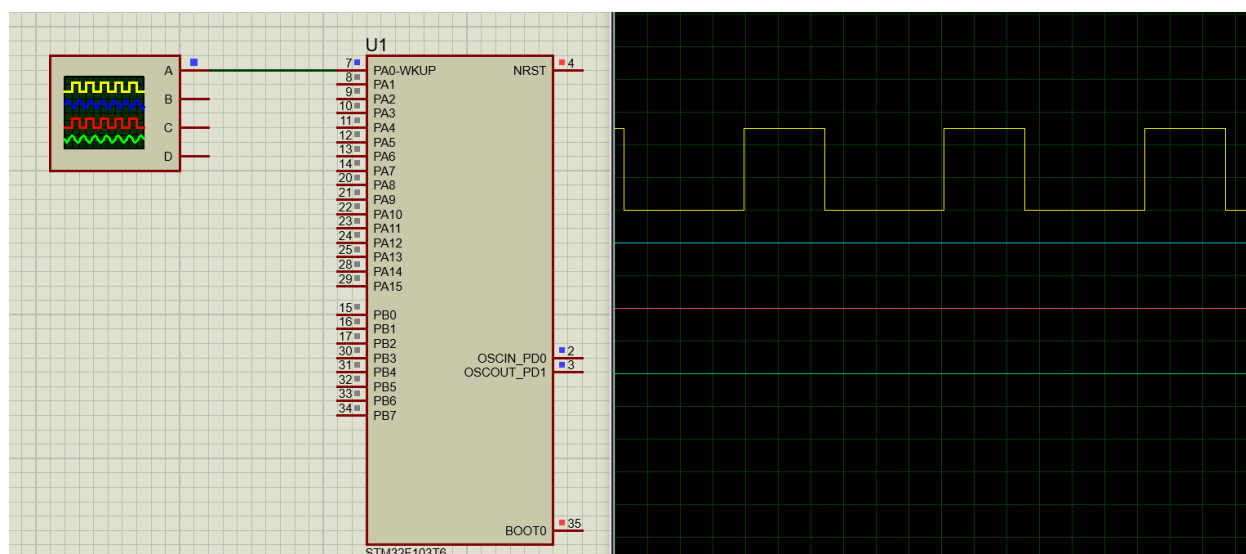
ج) با بستن مدار به طور خواسته شده در نهایت به شکل زیر مقدار ولتاژ سینوسی قابل مشاهده است. و طبق نکته گفته شده چون محدودیت 5 ولتی داریم که نباید شامل مقادیر منفی باشند، ما از سینوسی با اندازه 2 ولت و افست 3 ولت استفاده کردیم.



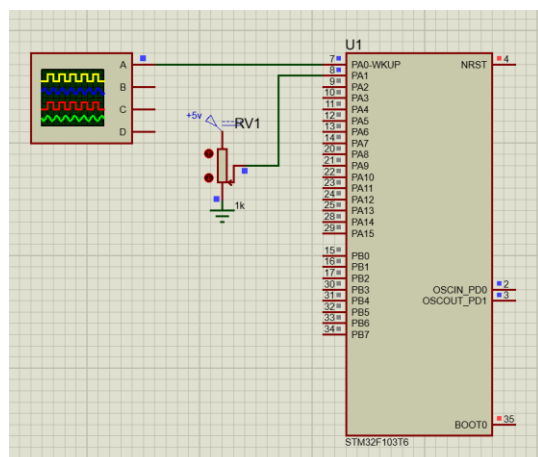
سوال (4)

الف) TIM در میکروکنترلر STM32 برای انجام کارهایی مانند شمارش زمان، تولید سیگنال PWM، شمارش ورودی خارجی، و ایجاد وقفه‌های دوره‌ای استفاده می‌شود. این تایمر دارای ویژگی‌هایی مانند Prescaler برای کاهش سرعت کلاک، شمارنده‌ای که تا مقدار Auto-Reload Register (ARR) می‌شمارد و قابلیت تولید وقفه‌ها یا خروجی PWM است.

برای اضافه کردن TIM، ابتدا به بخش Peripherals می‌رویم و TIM2 را انتخاب می‌کنیم. سپس حالت کانال را روی PWM Generation تنظیم می‌کنیم، Prescaler و Period را به ترتیب برابر 99 و 999 برای تعیین فرکانس PWM که برابر 100 است تنظیم کرده و Duty Cycle را از طریق CCR که برابر 400 گذاشته ایم کنترل می‌کنیم. در نهایت، کد تولید شده را با تابع HAL_TIM_PWM_Start اجرا می‌کنیم.



ب) در ادامه خواسته شده تا به کمک یک پایه ADC و پتانسیومتر، دیوتری سایکل را تغییر دهیم. برای این کار از یک پین را مثل PA1 برای ورودی ADC تنظیم می‌کنیم سپس با اضافه کردن کد زیر آن را تعریف می‌کنیم و مدار را بصورت زیر می‌بندیم:



```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM2_Init();
MX_ADC1_Init();

/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
uint16_t adc_val;
float duty_percent;
uint32_t ccr_val;
/* USER CODE END 2 */

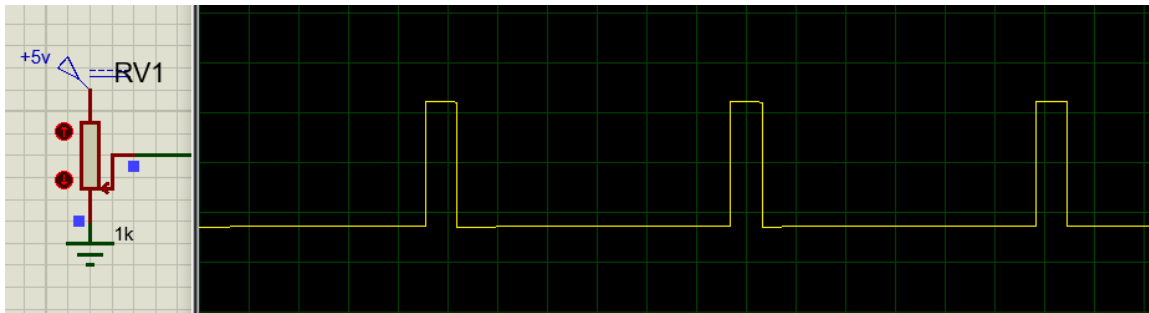
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    TIM2->CCR1 = 400;
    /* USER CODE END WHILE */
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    adc_val = HAL_ADC_GetValue(&hadc1);

    duty_percent = ((adc_val / 4095.0f) * 80.0f) + 10.0f;

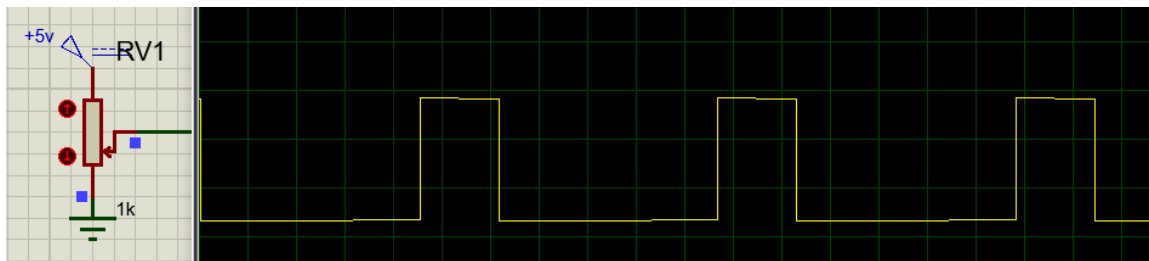
    ccr_val = (duty_percent / 100.0f) * (_HAL_TIM_GET_AUTORELOAD(&htim2) + 1);
    _HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, ccr_val);

    HAL_Delay(50);
    /* USER CODE BEGIN 3 */
}
```

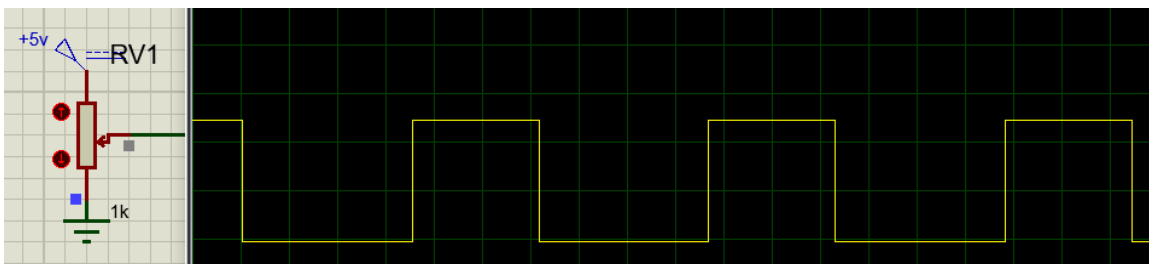
10%



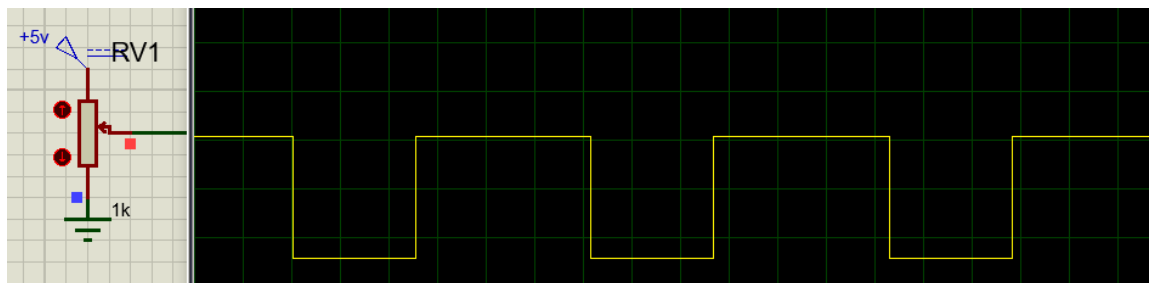
30%



50%



70%



90%

